

รายงานการวิเคราะห์และคำตอบ 7 ข้อ (Activity 7: Storage System Performance)

ชื่อ-นามสกุล: Phaolap Kulteera

รหัสนิสิต: 6630199021

รายวิชา: 2110415 Software-Defined Systems

1. ภาพรวมการทดลองและหลักฐานเชิงประจักษ์ (Empirical Evidence)

การทดลองนี้เปรียบเทียบประสิทธิภาพของระบบจัดเก็บข้อมูล 3 ระบบบน AWS:

1. **AWS EBS (gp3)**: Network-attached POSIX Block Storage
2. **EC2 Ephemeral Storage (Instance Store)**: Direct-attached NVMe SSD POSIX Storage บนอินสแตนซ์ c6gd.medium (ARM64)
3. **Amazon S3**: Cloud Object Storage ที่เข้าถึงผ่าน HTTP/HTTPS REST API (จัดการผ่าน Python SDK: boto3)

ผลการทดลองทั้งหมดถูกบันทึกอย่างสมบูรณ์ลงในตาราง [2110415 Storage Benchmark Template.xlsx](#) และไฟล์ [benchmark_results.json](#)

2. คำตอบสำหรับคำถามทั้ง 7 ข้อ (Questions & Comprehensive Analysis)

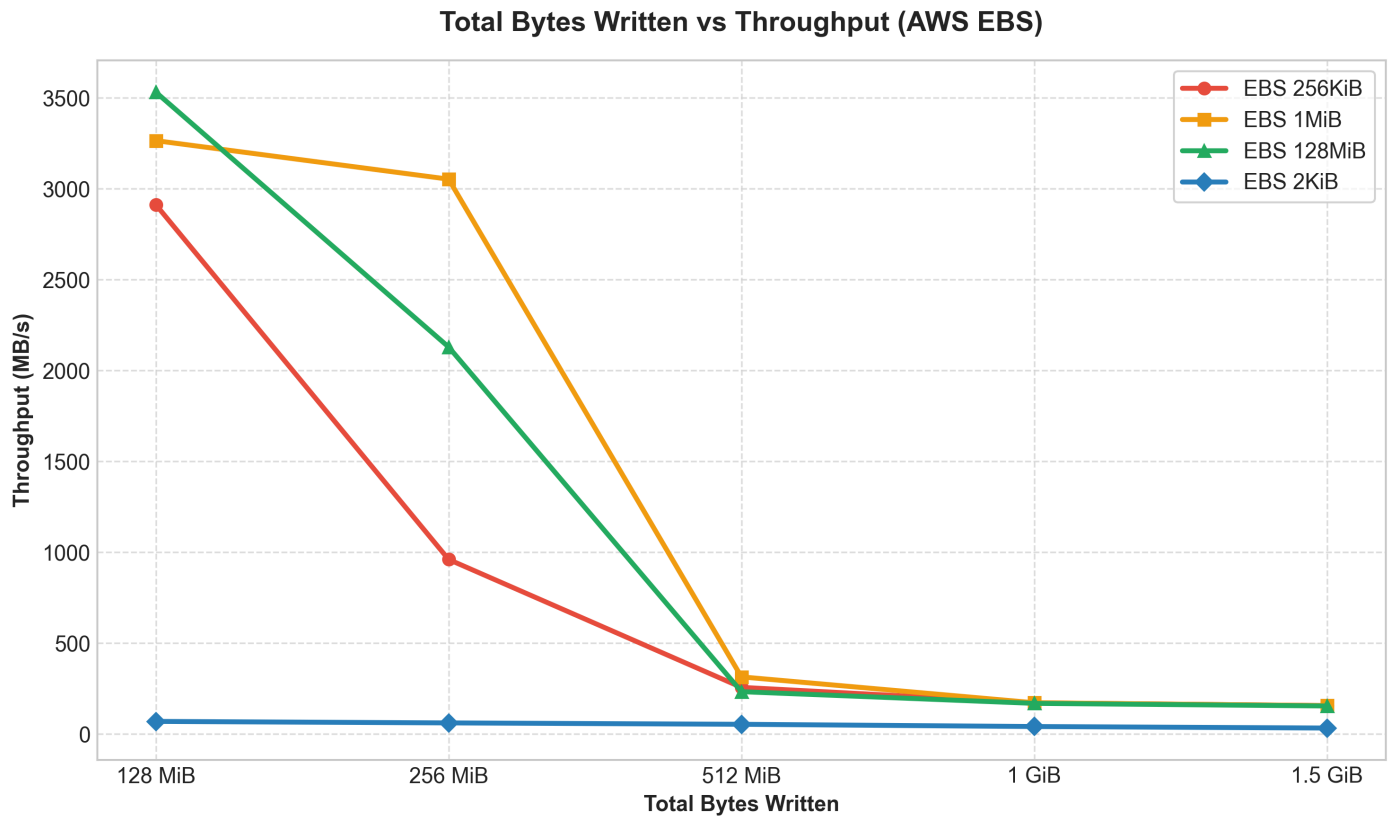
Question 1: Throughput vs. File Size

โจทย์: First, understand how each storage system performs with respect to file size. Does the throughput for EBS, Ephemeral storage and S3 vary with file size? Why or why not? Include the graph of your results. Hint: Normalize all experiments with Total Bytes Written so they are easy to compare.

คำตอบและบทวิเคราะห์:

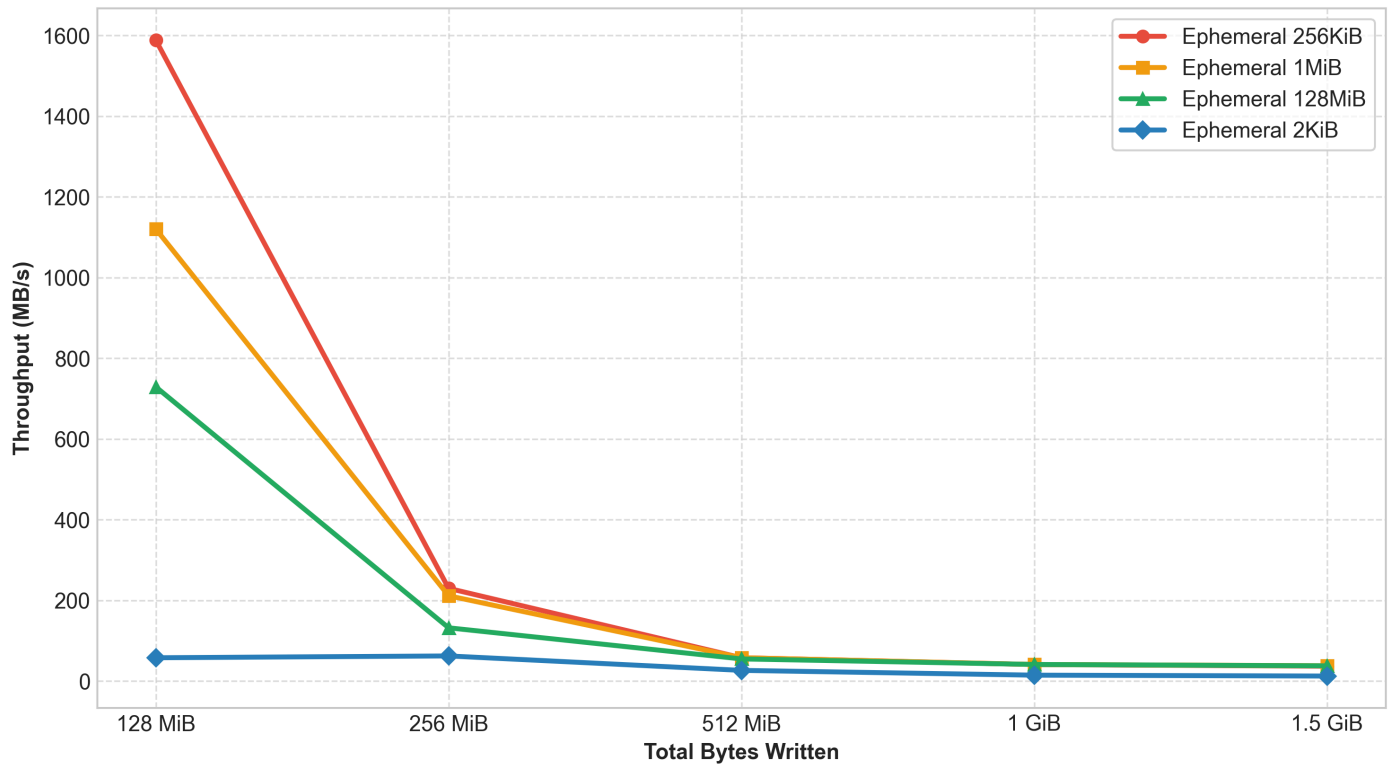
1. กราฟ Total Bytes Written vs Throughput สำหรับแต่ละระบบจัดเก็บข้อมูล (ตรงตาม Format ของอาจารย์ใน PDF หน้า 2)

- **AWS EBS (gp3):** มีครบทั้ง 4 ขนาดไฟล์ (2 KiB, 256 KiB, 1 MiB, 128 MiB)



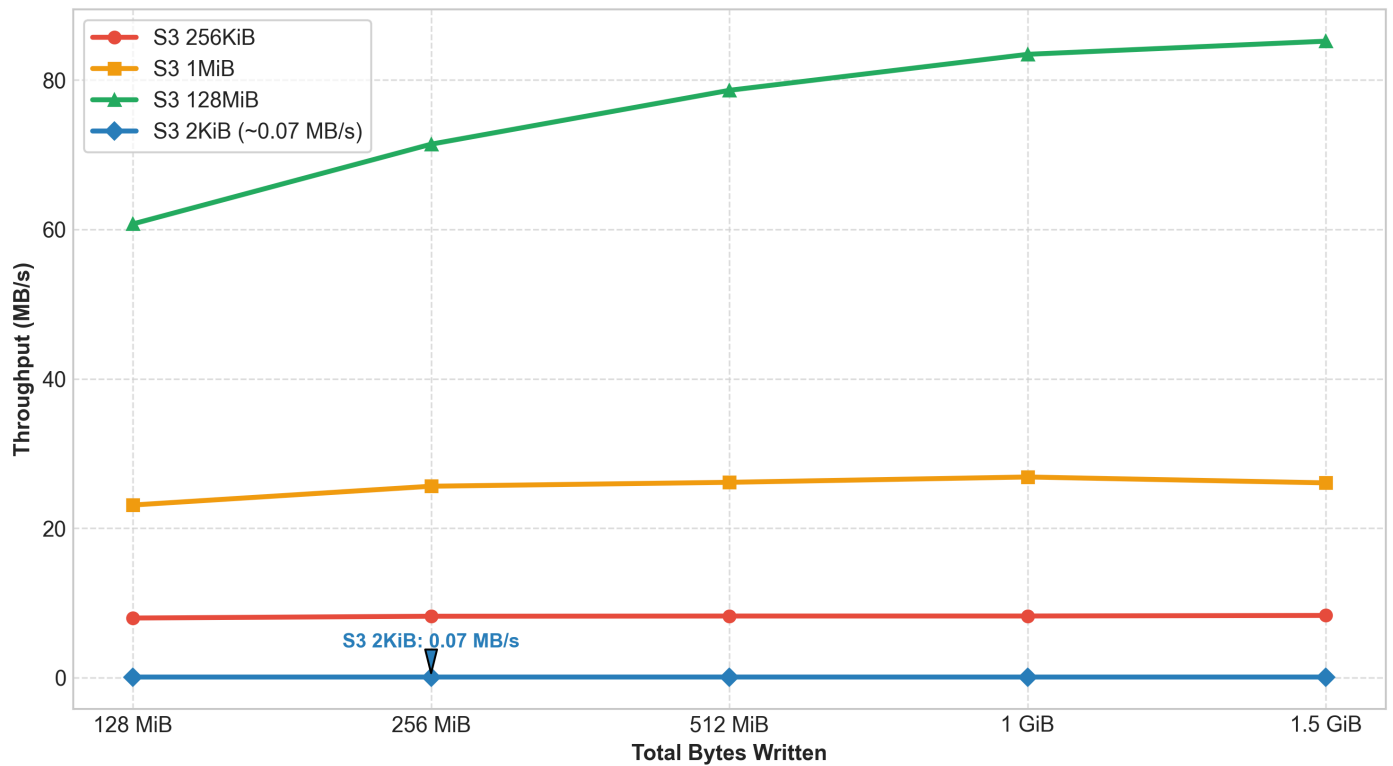
- **EC2 Ephemeral Storage (NVMe):** มีครบทั้ง 4 ขนาดไฟล์

Total Bytes Written vs Throughput (Ephemeral Storage NVMe)



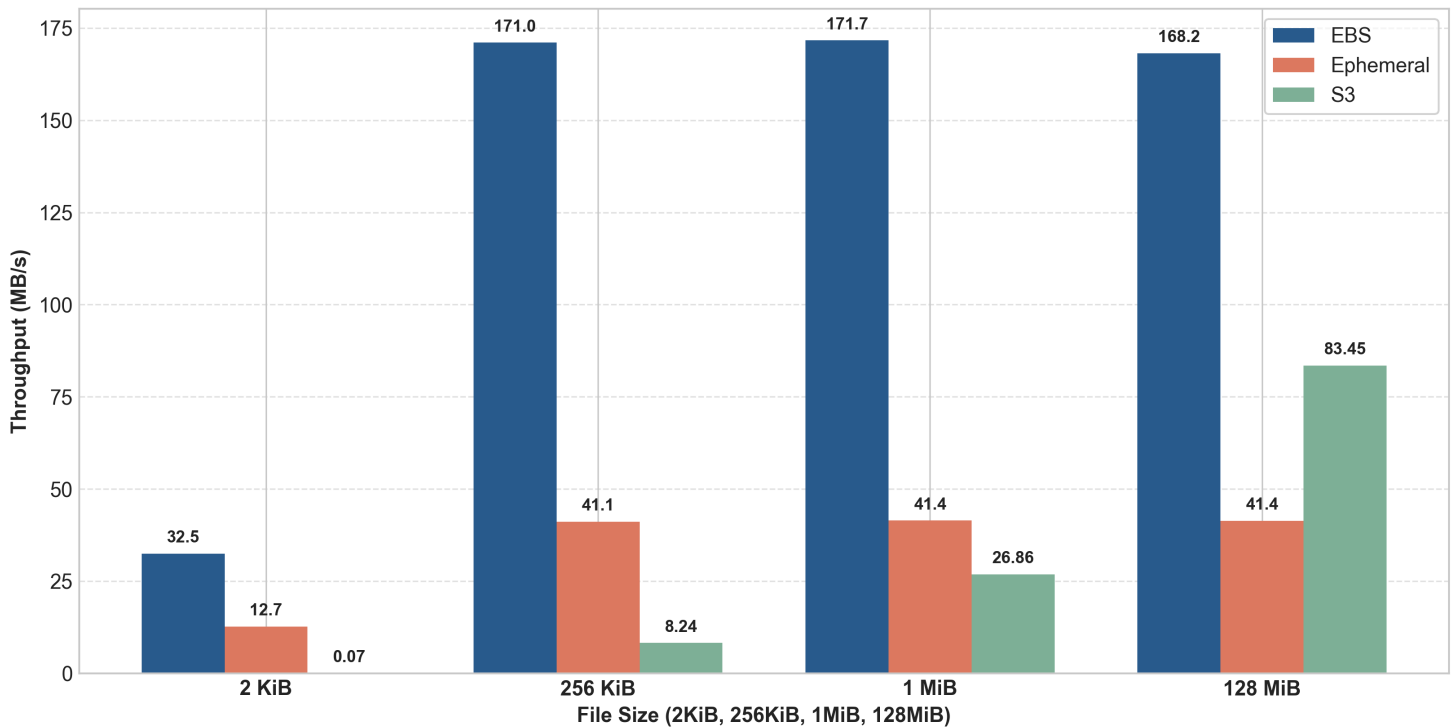
- **Amazon S3:** มีครบทั้ง 4 ขนาดไฟล์ (รวม S3 2KiB ซึ่ง Throughput อยู่ที่ ~0.07 MB/s)

Total Bytes Written vs Throughput (Amazon S3)



2. กราฟเปรียบเทียบทั้ง 3 Storage Tiers ครบทั้ง 4 ขนาดไฟล์ (Normalized Throughput Comparison)

Graph 1: Normalized Storage Throughput vs. File Size (All Storage Types)



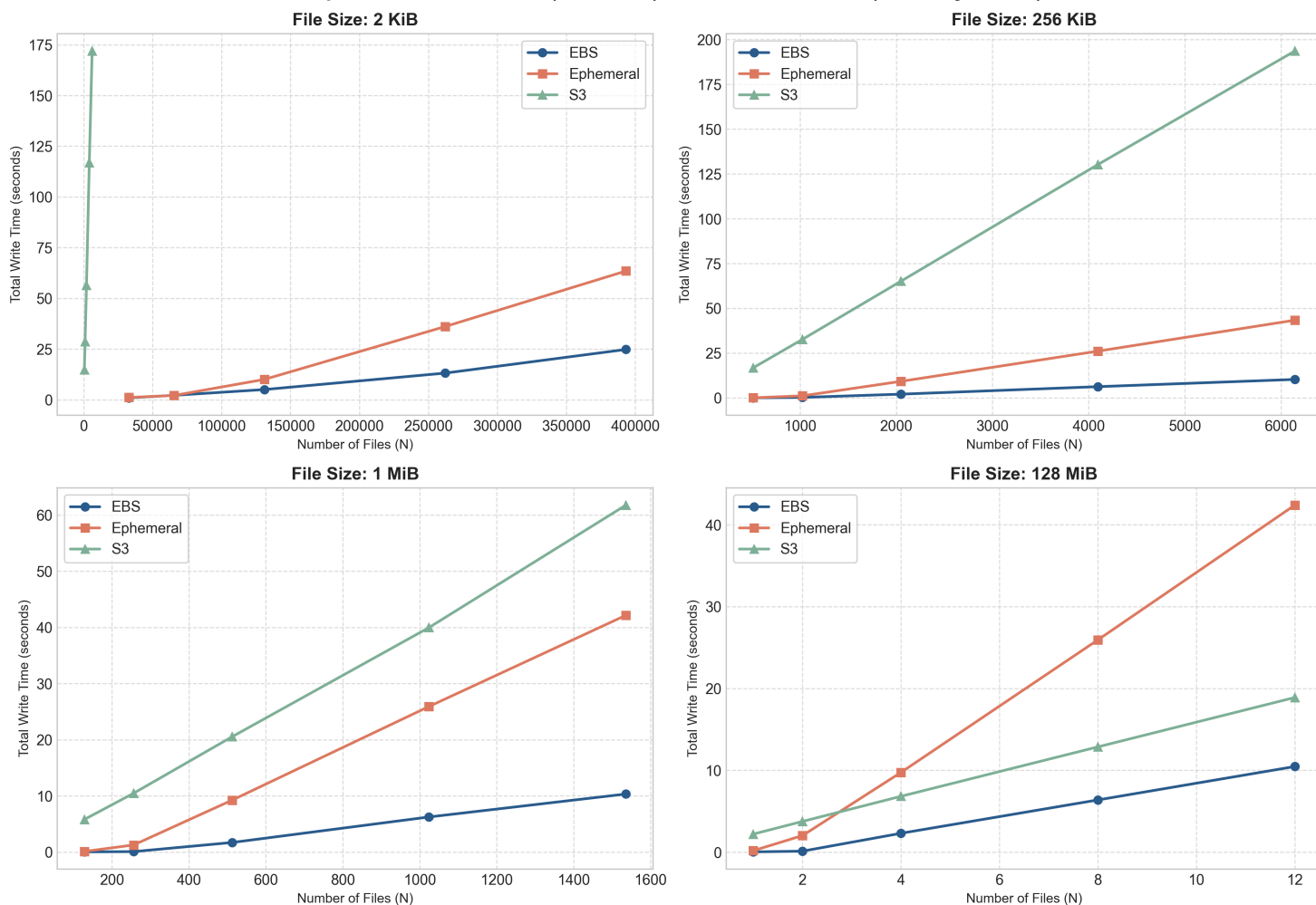
Answer: จากผลการทดลองจะได้ว่าที่ขนาดไฟล์ 2KiB: จะได้ผลว่า EBS, Ephemeral ที่ Iteration = 393216 ได้ throughput เป็น 32.46 MB/s และ 12.6862 MB/s และ S3 ที่ iteration = 6144 ได้ throughput เป็น 0.07 MB/s และที่ขนาดไฟล์ 256 KiB จะได้ผลว่า EBS, Ephemeral, S3 ที่ iteration = 6144, 6144, 6144 เป็น 156 MB/s, 37.13 MB/s, 8.3196 MB/s และที่ขนาดไฟล์ 1MiB จะได้ผลว่า EBS, Ephemeral, S3 ที่ iteration = 1536, 1536, 1536 เป็น 155.72 MB/s, 38.2 MB/s, 26.1 MB/s และที่ขนาดไฟล์ 128MiB จะได้ผลว่า EBS, Ephemeral, S3 ที่ iteration = 12, 12, 12 เป็น 153.88 MB/s, 38 MB/s, 85.2 MB/s ซึ่งจะเห็นว่า throughput ของทั้ง Ephemeral และ EBS คงที่เมื่อขนาดไฟล์มีขนาด 256KiB เป็นเพราะว่า throughput ขน gap ที่กำหนดแล้วเช่น Ephemeral cache ที่กักเก็บ byte ของไฟล์เต็มทำให้ความเร็วในการเขียนลง disk ใช้เต็ม speed ในการ write ส่วน EBS ก็ถึง cap ที่กำหนดจาก gp3 ที่กำหนดไว้ที่ 150 MB/s ทำให้เริ่มคงที่เพราะ throughput มากกว่านี้ไม่ได้ ส่วน S3 เมื่อขนาดไฟล์เล็กทำให้ต้องส่ง http request เป็นจำนวนมากส่งผลทำให้เวลาที่ใช้ไปในการส่งใช้เวลาไปกับ Network Latency และ Round trip time ทำให้ใช้เวลานานส่งผลทำให้ throughput ต่ำ ขณะเดียวกันเมื่อขนาดไฟล์ใหญ่ขึ้นเมื่อเทียบกับไฟล์ขนาดเล็กเปรียบเสมือนส่งไฟล์ได้ขนาดใหญ่ขึ้นขณะเสีย Round trip time และ Network latency แค่ครั้งเดียวเมื่อเทียบกับไฟล์ขนาดเล็กที่เสีย Network Latency และ Round trip time หลายรอบ อีกทั้งที่ขนาดไฟล์ใหญ่ก็เสีย TCP Handshake จำนวนครั้งน้อยลงและ Overhead จาก Header ของ HTTP ก็น้อยลงด้วย ซึ่งทำให้เวลาที่ใช้จริงใช้ไปกับการส่ง Byte ของเนื้อหาไฟล์จริงๆไม่ใช่ Overhead จาก Header หรือ Network latency ต่างๆ

Question 2: Iterations vs. Performance (Linearity)

โจทย์: Second, understand how each storage system performs with respect to the number of files (iterations) it must handle. Does the performance for EBS, Ephemeral storage and S3 scale linearly with the number of files? Why or why not? Include a graph of your results.

คำตอบและบทวิเคราะห์:

Graph 2: Number of Files (Iterations) vs. Total Write Time (Linearity Check)



Answer: สำหรับ S3 จะ Linearly ตลอดช่วงเวลาไม่ว่าจะ Iteration จะเท่าไรก็ตาม กลับกัน EBS และ Ephemeral จะ ไม่ Linearly ในช่วงแรกแต่เมื่อ Iteration หรือ จำนวนไฟล์ มากขึ้นจะ scale linearly โดยเหตุผลมาจากการที่ S3 เป็นการเขียนไฟล์ ผ่าน Protocol HTTP Request ซึ่งแต่ละ write เป็น Independent กันทำให้การเขียนไฟล์แต่ละครั้งมันใช้เวลาที่คูณตามจำนวนไฟล์ โดย จะเห็นว่าเมื่อ Iteration มากขึ้นกราฟของ S3 ทุก ขนาดไฟล์จะ Linear ตัวอย่างเช่นที่ 2KiB write time perfile จะประมาณ 0.028 s ทุก iteration เลย แต่ EBS และ Ephemeral จะใช้เวลาเร็วมากเมื่อจำนวนไฟล์น้อยๆเพราะ Cache ยังไม่เต็ม โดย Cache นี้คือ Linux Page Caching ซึ่งเมื่อ Cache ยังไม่เต็มทำให้เวลาที่ใช้ในการเขียน Cache จะเร็วมากแต่เมื่อ Iteration เพิ่มขึ้น Cache จะเริ่มเต็มทำให้ ข้อมูลที่อยู่ใน Cache ถูกบังคับเขียนลง Storage ทำให้เวลาที่ใช้มากขึ้นส่งผลให้ throughput กลายเป็น throughput ที่ disk รอง

รับจริงๆทำให้ N มากๆเวลาที่ใช้แปรผันตรงกับจำนวนไฟล์หรือ Iterationf ทำให้ช่วงหลังจากผ่านช่วงที่Cache เต็มจะ Scale linearly

Question 3: Compare EBS vs. Ephemeral Storage

โจทย์: Compare EBS vs. Ephemeral storage: 3.1. Which is faster? 3.2. Try to explain why. Focus on aspects related to their software-based implementations and protocols, beyond just the hardware. Look also at how the performance differs for different file sizes. Does this give you any insight as to what is going on?

3.1 Which is faster?

Answer: EBS เร็วกว่า Ephemeral โดยอ้างอิงจากผลลัพธ์กราฟในข้อ 2 แต่ Ephemeral ยังเร็วกว่าในแง่ของ Latency ระดับต่ำในการเข้าถึง Disk โดยตรง

3.2 Try to explain why. Focus on aspects related to their software-based implementations and protocols, beyond just the hardware. Look also at how the performance differs for different file sizes. Does this give you any insight as to what is going on?

Answer: เพราะ EBS มี Hardware เฉพาะที่ทำหน้าจัดการ EBS และมีช่องสัญญาณ Network พิเศษ ที่ส่งข้อมูลจาก Instance ไปยัง Cluster ที่เก็บ EBS ซึ่งถูก Optimize มาเฉพาะด้านโดยส่งผ่านเครือข่ายความเร็วที่สูงมทาก ในการรับข้อมูลทำให้ Throughput จะสูงมาก และ gp3 ก็กำหนด cap ของ throughput ไว้มากพอทำให้สามารถแสดง throughput ตามประสิทธิภาพของ Hardware ได้อย่างเต็มที่ ที่ประมาณ 150MB/s เมื่อเทียบกับ Ephemeral ที่พึ่งพาการเขียนไฟล์ลง Disk ที่อยู่ติดกับ Server Instance นั้นเลยแม้จะมีการพึ่งพา Cache ในช่วงแรกแต่เมื่อ Iteration มากขึ้น Cache เต็มจะเห็นชัดเจนว่า Throughput แ่ลงเพราะ ความเร็วในการเขียน Cache ไม่มีผลโดยถูกกำหนด Throughput จากกาเรเขียนลง disk โดยตรงซึ่งเป็น Bottle neck และเมื่ออ้างอิงจาก Performance เมื่อขนาดไฟล์เปลี่ยนไป จะเห็นว่าไม่ว่าขนาดของไฟล์จะเท่าไร EBS จะเร็วกว่า Ephemeral เสมอด้วย throughput ที่สูงกว่า แต่ที่ 2KiB ทั้ง Ephemeral และ EBS ทำ throughput ได้ต่ำกว่า 256KiB, 1Mib, 128MiB เป็นเพราะ ดิติดคอขวดที่ Inode Allocation และ Metadata Journaling ของระบบไฟล์ ext4 เอง

Question 4: Compare EBS vs. S3

โจทย์: Compare EBS vs. S3: 4.1. Which is faster? 4.2. Try to explain why. Focus on aspects related to their software-based implementations and protocols, beyond just the hardware. Look also at how the performance differs for different file sizes. Does this give you any insight as to what is going on?

4.1 Which is faster?

Answer: EBS เร็วกว่า S3 ในทุกกรณีแต่เมื่อไฟล์ ขนาดใหญ่มากขึ้นที่ 128MiB จะเห็นว่า throughput ของ S3 เป็น 85.2 MB/s ซึ่งเป็นครึ่งหนึ่งของ EBS ที่ขนาดไฟล์เดียวกัน

4.2 Try to explain why. Focus on aspects related to their software-based implementations and protocols, beyond just the hardware. Look also at how the performance differs for different file sizes. Does this give you any insight as to what is going on?

Answer: เพราะ EBS เป็น block store ซึ่งเป็นการเขียน block ลง disk ผ่าน Kernel โดยตรง แต่ S3 เป็น Object storage ที่ใช้การติดต่อการ write read ผ่าน Network protocol HTTP ซึ่ง throughput ที่ใช้เสียไปจาก Network Latency, TCP Handshake, การจัดการ Overhead ของ Header HTTP Protocol ซึ่งใช้เวลานานมากเมื่อเทียบกับการจัดการไฟล์ผ่าน EBS ที่ทำผ่าน Kernel อีกทั้ง S3 การันตี durability 99.99999% ซึ่งมีการ Peer to Peer ไปยัง 3 AZ ทำให้เมื่อมีการ Update file ใน AZ หนึ่งก็จะ update ไปยัง อีก 2 AZ ที่เหลือทำให้ใช้เวลานานมากขึ้นในการ Update file 1 ครั้ง

Question 5: S3 Partial Updates

โจทย์: Can an S3 object be updated without completely replacing the whole object? Why do you think AWS implemented it that way?

Answer: ไม่ได้เพราะ S3 ถูกออกแบบภายใต้แนวคิด Write One Readmany ถ้าเกิดอนุญาตให้ update บางส่วนได้ การทำ Cache ข้อมูลจะทำได้ยากขึ้นเพราะมีการ write และ update หลายครั้ง รวมถึง S3 จัดเก็บข้อมูลแบบกระจายถ้า update บางส่วนการทำ synchronization จะทำได้ยากขึ้นในหลาย AZ ทำให้ซับซ้อนขึ้นมหาศาล

Question 6: S3 Permissions & Least Privilege JSON

โจทย์: What permissions did you use to access S3? Explain and include your policy as a JSON in your answer. (Hint: You should NOT use AmazonS3FullAccess. You should use least privileged for good security practice.)

Answer: ใช้หลักการ Least Privilege โดย Json file เป็นไปดังนี้

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeastPrivilegeS3BenchmarkAccess",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::act7-storage-benchmark-*",
        "arn:aws:s3:::act7-storage-benchmark-*/*"
      ]
    }
  ]
}
```

คำอธิบายสิทธิ์ทั้ง 4 ตัวที่จำเป็นสำหรับการทดลอง:

1. s3:PutObject : อนุญาตให้อัปโหลดสร้าง Object ในการทดสอบ Benchmark (test_s3.py)
2. s3:GetObject : สิทธิ์พื้นฐานสำหรับตรวจสอบความถูกต้องของข้อมูลอบเจกต์
3. s3:DeleteObject : จำเป็นสำหรับการลบไฟล์ทดสอบเก่าออกจากรอบการทดลอง (Auto-Cleanup) และการสั่ง Empty Bucket
4. s3:ListBucket : อนุญาตให้ดูรายการไฟล์ในบั๊กเก็ตเพื่อตรวจสอบและค้นหาไฟล์ที่จะลบ

- **การจำกัดขอบเขต Resource:** จำกัดสิทธิ์ให้มีผลเฉพาะบั๊กเก็ตที่ขึ้นต้นด้วย act7-storage-benchmark-6630199021 เท่านั้น เพื่อป้องกันไม่ให้เกิดกระทบต่อบั๊กเก็ตอื่น

Question 7: S3 Optimization Bottleneck

โจทย์: If your only goal is to write all n files to S3 as fast as possible, how would you optimize/change your code? Suggest at least 1 optimization. (Hint: What is the bottleneck for S3?)

Answer: โดย bottle neck ของ S3 คือ Network Latency, Round trip time และอื่นๆ โดยหลักๆเกิดจาก Round trip time โดย code ปัจจุบันผมทำงานแบบ synchronous วนลูปทีละ iteration ต้องเสีย 1 round trip time ทุกครั้งที่เขียนไฟล์เสร็จ 1 รอบหรือกล่าวคือเสีย 1 RTT ต่อ 1 iteration

ส่งไฟล์ที่ 1 → รอ RTT 28ms → ส่งไฟล์ที่ 2 → รอ RTT 28ms ...

ทำให้ท่อสื่อสารของเครือข่ายส่วนใหญ่ปล่อยว่าง (Idle) ในระหว่างรอการตอบกลับจากเซิร์ฟเวอร์ S3

วิธีการ Optimize สามารถทำได้โดยการใช้เรื่องของ Multi thread โดยการสร้าง thread มาช่วยในการ send http request พร้อมๆกันโดยสาเหตุที่ทำได้เพราะแต่ละไฟล์ไม่เกี่ยวข้องกันไม่ต้องสนใจเรื่อง ลำดับการรับของ S3 ทำให้เมื่อใช้ thread ช่วยจะทำให้ เครือข่ายสามารถใช้ประโยชน์ได้อย่างเต็มที่เพราะระหว่างที่ thread 1 idle thread 2 ก็สามารถส่งได้ทันทีโดยไม่ต้องรอ หรือถ้าไฟล์ที่ส่งมีความเกี่ยวข้องกันสามารถใช้ S3 Multi Upload ได้เพื่อแบ่งไฟล์เป็นส่วนย่อยๆและใช้เรื่อง multithread ในการส่งไฟล์